МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНОМУ УНІВЕРСИТЕТІ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”

Кафедра систем штучного інтелекту

Лабораторна робота №12

з дисципліни
«Операційні системи»

Виконав:
студент групи КН-114
Добрій Назарій

Викладач:
Кривенчук Ю.П.

Львів – 2020 р.

Тема. Файли, що відображаються в пам'ять в ОС Linux.

Мета. Ознайомитися з відображенням файлів в оперативну пам’ять в ОС Linux. Навчитися реалізовувати відображення файлів в оперативну пам’ять.

Завдання.

1. Реалізувати відображення файлу в оперативну пам'ять.
2. Необхідно модифікувати **лабораторну роботу №10** таким чином, щоб всі результати роботи програми записувалися через відображення файлів в оперативній пам’яті.
3. Порівняти результати виконання програм під ОС Windows та Linux.
4. Результати виконання роботи відобразити у звіті.

Код реалізації програми:

```
#include <pthread.h>
#include <iostream>
#include <vector>
#include <ctime>
#include <sys/mman.h>
#include <sys/types.h>
#include <unistd.h>
#include <fcntl.h>
#include <string>

#define FILESIZE 256

using namespace std;

string res = "";

pthread_t TIDs[8];

int done = 0;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

void thr_exit() {
    pthread_mutex_lock(&mutex);
    done = 1;
    pthread_cond_signal(&cond);
    pthread_mutex_unlock(&mutex);
}

void thr_join() {
    pthread_mutex_lock(&mutex);
    while (done == 0) {
        pthread_cond_wait(&cond, &mutex);
    }
    pthread_mutex_unlock(&mutex);
}

void* selectionSort(void* arg)
{
    vector<int>& temp = *(vector<int>* ) arg;
```

```
    int min = 0;
    for (int i = 0; i < temp.size() - 1; i++) {
        // start critical section
        pthread_mutex_lock(&mutex);

        min = i;
        for (int j = (i + 1); j < temp.size(); j++)
            if (temp[j] < temp[min])
                min = j;
        swap( &temp[i], &temp[min]);

        // end critical section
        pthread_mutex_unlock(&mutex);
    }

    thr_exit();
    return NULL;
}


vector<int> complete(vector<vector<int>> arr, int n, int k) {
    vector<int> completed(n);
    for (int i = 0; i < n; ++i) {
        int min = 1000000;
        int th = -1;
        for (int j = 0; j < k; j++) {
            if (!arr[j].empty() && arr[j][0] < min) {
                min = arr[j][0];
                th = j;
            }
        }
        completed[n - i - 1] = min;
        arr[th].erase(arr[th].begin());
    }
    return completed;
}

void write_to_file(vector<int>arr, void* f){
    for (int i = 0; i < arr.size(); ++i) {
        res += to_string(arr[i]);
        res += " ";
```

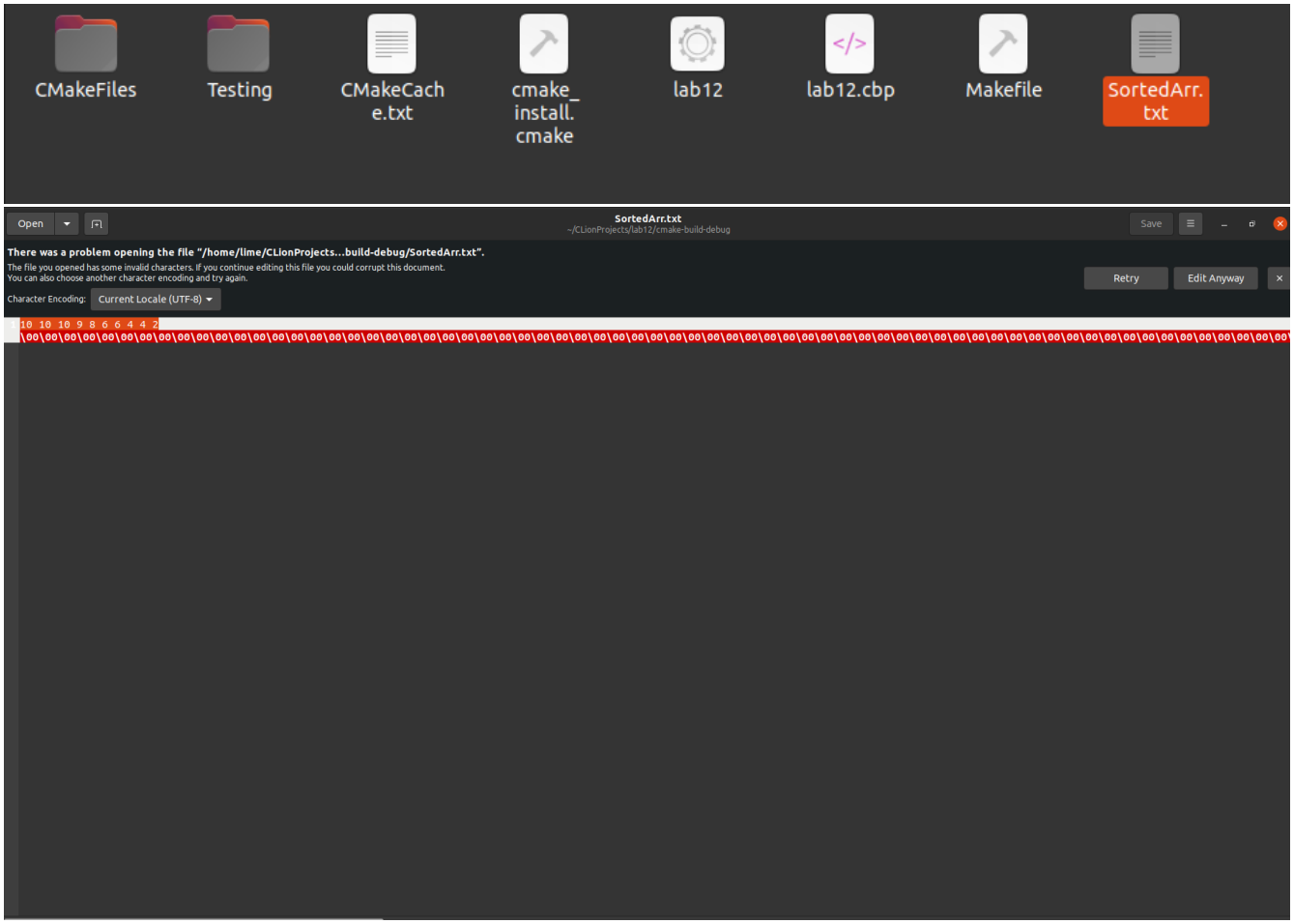
```
    f

    const char* c_res = res.c_str();

    sprintf((char*)f, format: "%s", c_res);
}

int main()
{
    srand( seed: time( timer: NULL));

    int fd;

    void* file_memory;

    fd = open( file: "SortedArr.txt", oflag: O_RDWR | O_CREAT, S_IRUSR | S_IWUSR);

    int amount_num, amount_thread;
    vector<vector<int>> arr;

    cout << "Enter the number of elements: ";
    cin >> amount_num;
    vector<int> input(amount_num);
    for (int i = 0; i < amount_num; i++) {
        input[i] = rand() % amount_num + 1;
        cout << input[i] << ' ';
    }
    cout << endl;

    cout << "Enter the number of threads: ";
    cin >> amount_thread;
    for (int i = 0; i < amount_thread; i++) {
        vector<int> tmp;
        for (int j = (i * amount_num / amount_thread); j < (i + 1) * amount_num / amount_thread; j++) {
            tmp.push_back(input[j]);
        }
        arr.push_back(tmp);
    }

    // Launch thread
    for (int i = 0; i < amount_thread; i++) {
        vector<int> temp = arr[i];
```

```
        vector<int> temp = arr[i];

        pthread_attr_t attr;
        pthread_attr_init(&attr);

        pthread_create(&TIDs[i], &attr, selectionSort, &arr[i]);

        cout << "TID = " << TIDs[i] << endl;
        cout << "Taken arguments:" << endl;
        for (auto now : temp) {
            cout << now << " ";
        }
        cout << endl;
    }

    // Wait until thread is done its work
    for (int i = 0; i < amount_thread; i++) {
        pthread_join(TIDs[i], thread_return: NULL);
    }

    vector<int> completed = complete(arr, amount_num, amount_thread);
    cout << "Sorted array: " << endl;
    for (int i = 0; i < amount_num; ++i) {
        cout << completed[i] << ' ';
    }
    cout << endl;

    lseek(fd, FILESIZE, SEEK_SET);

    write(fd, buf: "", n: 1);

    lseek(fd, offset: 0, SEEK_SET);

    file_memory = mmap( addr: 0, FILESIZE, PROT_WRITE, MAP_SHARED, fd, offset: 0);

    write_to_file(completed, file_memory);

    munmap(file_memory, FILESIZE);

    char buffer[FILESIZE];
    read(fd, &buffer, FILESIZE);

    cout << "Data in buffer:" << endl;

    for (int i = 0; i < FILESIZE; i++) {
        cout << buffer[i];
    }

    close(fd);

    return 0;
}
```

Вивід результатів в console та ілюстрація даних, які знаходяться у буфері:

```
Run:  lab12
/home/lime/CLionProjects/lab12/cmake-build-debug/lab12
Enter the number of elements: 10
10 6 4 4 10 2 8 10 9 6
Enter the number of threads: 3
TID = 140494057129728
Taken arguments:
10 6 4
TID = 140494048737024
Taken arguments:
4 10 2
TID = 140494040344320
Taken arguments:
8 10 9 6
Sorted array:
10 10 10 9 8 6 6 4 4 2
Data in buffer:
10 10 10 9 8 6 6 4 4 2
Process finished with exit code 0
```

Запис даних у файл SortedArr.txt:



Висновок: я ознайомився з відображенням файлів в оперативну пам’ять в ОС Linux. Навчитися реалізовувати відображення файлів в оперативну пам’ять. Навчився виводити дані з буферу та за допомогою функції munmap() переписувати дані з буфера у створений файл.